

Epic: Make Dumb Copilot Smarter (MDCS)

This epic designs and implements the MDCS framework within this repository. The goal is to provide a local pre-processing toolset and a structured prompt library based on the **Aegis Progressive Disclosure (Closet/Drawer) architecture**. This setup optimizes the utility of a web-based, GPT-3.5-based Edge Copilot for an IT-Business Analyst working with Oracle SQL databases and Python scripts.

Architecture Overview

Because the target Copilot runs on GPT-3.5 inside a web browser, it suffers from a limited context window, lacks direct database access, and cannot handle real enterprise data.

To solve these constraints, we implement a two-pronged solution: 1. **Local Utilities (tools/)**: Python scripts run locally by the user to compress database schemas (reducing token count) and anonymize datasets (masking PII/sensitive data) before pasting them into Copilot. 2. **Progressive Disclosure Prompts: * Procedures (Closets under procedures/)**: High-level, action-oriented, step-by-step instructions. They are size-budgeted (**at most 200 lines**) and use structured **reference_pointers** to tell the user which drawer files to load next under specific conditions. * **References (Drawers under references/)**: Dense coding patterns, idioms, and database-specific knowledge blocks. They are indexed via a README.md containing a **Sections** table (columns: **id** | **topic** | **open when**, size-budgeted to **at most 120 lines**) and split into individual topical markdown files (drawers, size-budgeted to **at most 150 lines**) under **sections/**. Each drawer file begins with a one-line **relevant-when**: header (key-style line).

Intentional MDCS Adaptations from Aegis Doctrine

MDCS is a consumer prompt library designed for a single human analyst driving a web chat interface. Thus, we intentionally adapt the canonical Aegis architecture as a deliberate design choice: * **Root-Level Folders**: Instead of nesting procedures and references under a **skills/** directory (e.g., **skills/procedures/** and **skills/references/**), we place **procedures/** and **references/** at the root of the MDCS repository for direct, clean accessibility on the user's workspace. * **Two-Layer Scope**: MDCS implements only the **Procedural** and **Reference** layers of Aegis. The autonomous role layer (Master, Worker, Validator prompts) and the multi-agent operating discipline layers are omitted as out of scope, since the driving agent is a human analyst interacting with a single chat session.

Ticket List

Work is divided into the following tickets under **to-do/tickets/**. Each ticket defines a single atomic step with explicit boundaries, acceptance criteria, and verification commands:

Ticket ID	Title	Status	Depends On
MDCS-TICKET-001	Setup repository structure and local library validator	ready	(None)
MDCS-TICKET-002	Implement schema compressor utility	ready	MDCS-TICKET-001
MDCS-TICKET-003	Implement data anonymizer utility	ready	MDCS-TICKET-001
MDCS-TICKET-004	Create Oracle SQL reference drawers	ready	MDCS-TICKET-001
MDCS-TICKET-005	Create Python idioms reference drawers	ready	MDCS-TICKET-001
MDCS-TICKET-006	Create Query Tuner procedural closet	ready	MDCS-TICKET-004
MDCS-TICKET-007	Create Report Generator procedural closet	ready	MDCS-TICKET-005

Ticket ID	Title	Status	Depends On
MDCS-TICKET-008	Create Mock Data Generator procedural closet	ready	MDCS-TICKET-002, MDCS-TICKET-004

Allowed Repository Areas

The following directories are defined as active zones for this epic: - `tools/` - `procedures/` - `references/` - `to-do/`

All other areas, including the `.git/` folder and runtime configurations, must not be touched.